Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 3: Importing data from text files, XML and HTML

Dr. Aurélien Sallin (Lecturer) Ana Armendariz (Tutor)

15 October 2026

Table of contents

Before we start 1

Exercise 1: importing a CSV in two ways 2

Exercise 2: column types 2

Exercise 3: select what you need and write it back 3

Exercise 4: the same data in XML 3

Exercise 5: a file that is not comma separated 4

Exercise 6: what the browser shows and what the server sent 4

Additional exercises for training 5

Exercise 7: scraping a table from a web page 5

All course material: https://github.com/ASallin/BEcon_3230_Datahandling_ImportCleaningandVisualization

Before we start

These exercises follow lectures 4 and 5. We import one data set twice, out of a CSV and out of an XML document, then read a text file that is not comma separated, and finish with the data behind a web page.

Write your code in an R script saved in `2026_Data_Handling/exercises/code`, for example `exercise_03.R`. Open your RStudio project first, so that every path starting with `data/` works. Type the code rather than copying it, and read every error message. If something does not work, raise your hand.

Exercise	What we do
1	Importing a CSV in two ways
2	Column types: what the parser guessed, and how to correct it
3	Select what you need and write it back to disk
4	The same data in XML
5	A file that is not comma separated
6	What the browser shows and what the server sent
Training	Scraping a table from a web page

The data. We use three files: `Air_Quality.csv`, `Air_Quality.xml` and `temp_anomaly.txt`. Download them from the course repository, folder `materialsExercises/03_programming_data/data`, and save them in your own `data` folder.

The air-quality files are a snapshot of a data set published by the City of New York, listed at https://catalog.data.gov/dataset/air-quality. Use our copies: the version online is newer and would give different numbers from the solutions.

Exercise 1: importing a CSV in two ways

One row of `Air_Quality.csv` is one measurement of one air-quality indicator, in one part of the city, over one time period.

1. Import the file twice, once with base R and once with `readr`:

```
air_quality_base <- read.csv("data/Air_Quality.csv")

library(readr)
air_quality <- read_csv("data/Air_Quality.csv")
```

2. Inspect what you imported with `head()`, `tail()`, `dim()`, and `glimpse()` from `dplyr`. How many rows and columns are there?
3. Compare `names(air_quality_base)` with `names(air_quality)`. The file has a column called `Unique ID`, with a space in the name. What did each function do with that name, and how do you refer to the column afterwards?
4. Compare `class(air_quality_base)` with `class(air_quality)`. What is the difference between the two objects?
5. `read_csv()` printed a message about column specifications, `read.csv()` printed nothing. Which behaviour do you prefer when you open a file for the first time, and why?

Exercise 2: column types

`read_csv()` reads the first 1000 rows, guesses a type for every column, and reports what it guessed.

1. Print the full specification with `spec(air_quality)`. Which columns came out as `col_double()` and which as `col_character()`?
2. `Start_Date` contains dates such as `06/01/2013`, but it was not read as a date. Why can the parser not recognise it, and what would the file have to look like for `readr` to guess a date by itself?
3. State the type yourself instead of letting R guess:

```
air_quality_dates <- read_csv(
  "data/Air_Quality.csv",
  col_types = cols(Start_Date = col_date(format = "%m/%d/%Y"))
)
```

Check `class(air_quality_dates$Start_Date)`, then run `range()` on that column. Which period does the data cover? Run `range()` on the text version too, and explain the difference.

4. Why is it better to fix a type at import than to convert the column afterwards?

Exercise 3: select what you need and write it back

1. `Name` holds the indicator that was measured. List the distinct values with `unique(air_quality$Name)`. How many are there?
2. Keep the rows measuring fine particulate matter (`Name == "Fine Particulate Matter (PM2.5)"`) in the winter of 2008-09 (`Time Period == "Winter 2008-09"`) in lower Manhattan (`Geo Place Name == "Lower Manhattan"`). Use `subset()`, and link the conditions with `&`:

```
pm25_winter <- subset(air_quality,
                      Name == "Fine Particulate Matter (PM2.5)" &
                      `Time Period` == "Winter 2008-09" &
                      `Geo Place Name` == "Lower Manhattan")
pm25_winter
```

Note the backticks around the column names that contain a space. How many rows are left, and what is the measured value?

3. Save the result as `airquality_manhattan.csv` in your `data` folder with `write_csv()`, then open the file in a text editor and look at what was written.
4. Now keep every measurement of fine particulate matter in Lower Manhattan, for all time periods, and save it the same way. If you read `Start_Date` as a date in Exercise 2, sort the series by date first.

Exercise 4: the same data in XML

`Air_Quality.xml` holds the same measurements as the CSV, in a different format.

1. Load the file. The package to use here is `xml2`.

```
library(xml2)
doc <- read_xml("data/Air_Quality.xml")
class(doc)
xml_name(doc)
```

What kind of object did you get? Is it a data frame?

2. Every measurement sits in a `row` element inside the root. Collect them and count them:

```
rows <- xml_find_all(doc, "//row/row")
length(rows)
```

Compare the number with the number of rows in the CSV.

3. Inspect the first measurement: the names of its children, and their contents.

```
xml_name(xml_children(rows[[1]]))
xml_text(xml_children(rows[[1]]))
```

4. Extract one variable for all measurements with an XPath expression, for example every `data_value`. What type is the result, and what do you have to do before you can compute the mean?
5. Compare `start_date` here with `Start_Date` in the CSV, and the size of the two files with `file.size()`. What does XML give you that a CSV cannot, and what do you pay for it?

Exercise 5: a file that is not comma separated

`temp_anomaly.txt` holds a monthly series of surface temperature anomalies from NASA's GISTEMP analysis, for the coordinates of St. Gallen. Suppose you are writing a thesis on the effect of climate change on consumption behaviour in Switzerland, and you need this series as a proxy for the salience of climate change.

This exercise is adapted from the tutorial in Chapter 1 of Paul Murrell's book, see the syllabus. Read the chapter for a longer introduction to data stored in text files.

1. Before you do anything else, look at the file. Open `data/temp_anomaly.txt` in a text editor, or print its first lines from R:

```
readLines("data/temp_anomaly.txt", n = 12)
```

The first lines are not data. How many lines of metadata are there, and on which line does the first observation start?

2. Read the file:

```
anomalies <- read_tsv("data/temp_anomaly.txt",
                      skip = 9,
                      col_names = c("month", "temp_anomaly"))
head(anomalies)
```

3. What is the purpose of the `skip` argument, and why is it set to 9? And why does `col_names` have to be given here?
4. Why do we use `read_tsv()` and not `read_csv()`? Try it with `read_csv()` and look at the result: how many columns do you get, and what is inside them?
5. How many observations does the series contain, and what is the mean anomaly?

Exercise 6: what the browser shows and what the server sent

HTML is the code behind most websites, and it is also the data a scraper reads. We look at the page we scrape in Exercise 7.

1. Open https://en.wikipedia.org/wiki/Divided_government_in_the_United_States in your browser and find the table with the year, the president, and the majority party in the Senate and in the House.
2. Right-click on one cell of that table and choose *Inspect*, then find the table in the code. A table is `<table>`, a row is `<tr>`, a cell is `<td>`, and a header cell is `<th>`:

```
<table class="wikitable">
  <tr><th>Year</th><th>Senate</th><th>House</th></tr>
  <tr><td>1861</td><td>Republican</td><td>Republican</td></tr>
</table>
```

Which tags hold the years, and which hold the party names?

3. Scroll up in the *Inspect* panel until you reach the title of the page, which is an `<h1>` element. Double-click the text, type a title of your own, and press Enter. What happens on the screen?
4. Now read the same page with R and print its first heading:

```r
library(rvest)

doc <- read_html("https://en.wikipedia.org/wiki/Divided_government_in_the_United_States")
html_text(html_element(doc, "h1"))
```

Does your title appear? Why not?

5. Reload the page in the browser. Where did your edit go, and who else saw it?
6. What does this tell you about the difference between what a website shows and what the data behind it is?

Additional exercises for training

> **Note**
>
> Exercise 7 takes the step we did not have time for in the session: reading a table out of a web page and into R. Do it at home, the solution is published with the rest.

Exercise 7: scraping a table from a web page

In the lecture we saw that an HTML document is a semi-structured data source. This exercise has the basic steps for importing data from an HTML table into R.

The aim is a CSV file with data on divided government in US politics, from this Wikipedia page: https://en.wikipedia.org/wiki/Divided_government_in_the_United_States. The page has a table with the president's party and the majority party in the House and in the Senate per Congress, that is per two-year period. The first rows of the cleaned data should look like this:

```
# A tibble: 4 x 4
   year president senate house
  <dbl> <chr>     <chr>  <chr>
1  1861 Lincoln   R      R
2  1862 Lincoln   R      R
3  1863 Lincoln   R      R
4  1864 Lincoln   R      R
```

Start by fixing the paths and loading the packages:

```r
library(rvest)
library(dplyr)
library(stringr)
library(readr)

SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"
```

1. Fetch the whole HTML document from Wikipedia with `read_html()`.
2. Extract the tables with `html_table(fill = TRUE)`. The result is a list, so look at the candidates and find the one with `Year`, `President`, `Senate` and `House`. Store it as `tab`.
3. Check the last row. If it contains a footnote instead of data, remove it.

4. The column `Year` contains ranges such as `1861-1863`, but we want one row per year. Write a loop that turns each range into one row per year. `strsplit()` cuts a string into pieces, `unlist()` flattens the result, and `bind_rows()` stacks the pieces together. See `?strsplit` and this introduction to regular expressions for the background.
5. Remove Wikipedia's footnote markers, `[a]`, `[b]` and so on, with `str_remove_all(x, "\\[[^\\]]*\\]")`.
6. Inspect the result and write it to a CSV file, using `OUTPUT_PATH`.